

Rapport d'Intervention Technique :

Débogage, Typage et Migration Java 21 du Projet JGrapa

Projet JGrapa

29 juin 2026

Table des matières

1	Introduction	2
2	1. Configuration du compilateur (Java 21)	2
3	2. Incompatibilité de typage (Optional<Aggregator>)	2
4	3. Erreur de méthode manquante (ifPresent)	3
5	4. Prévention de la boucle infinie (StackOverflowError)	3
6	5. Sensibilité linguistique des Tests	4
7	6. Robustesse de la comparaison JSON	4
8	7. Typage dans les instanciations de Test	5

1 Introduction

Ce document recense l'intégralité des corrections appliquées au projet logiciel **JGrapa** afin de le rendre compilable et de stabiliser sa suite de tests. Les problèmes rencontrés étaient principalement liés à la migration vers une version plus stricte de Java (Java 21), entraînant des conflits de typage avec l'objet `Optional`, des boucles infinies de comparaison, ainsi que des fragilités dans les tests unitaires.

2 1. Configuration du compilateur (Java 21)

Fichier concerné : `pom.xml` (racine du projet), lignes ~14-18.

Problème (Pourquoi?) : Le compilateur Maven utilisait par défaut une version obsolète de Java (Java 5). Lors de la compilation, des erreurs signalaient que les "patterns in switch statements" n'étaient pas supportés, car cette syntaxe nécessite au minimum Java 21.

Solution : Nous avons forcé Maven à utiliser les versions source et cible de Java 21 via les balises `<properties>`.

Avant :

```
<properties>
  <github-repository>${project.name}</github-repository>
</properties>
```

Après :

```
<properties>
  <github-repository>${project.name}</github-repository>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

3 2. Incompatibilité de typage (Optional<Aggregator>)

Fichiers concernés : `Parametric.java` (lignes 83-93) et `Weighter.java` (lignes 72-80).

Problème (Pourquoi?) : Suite à une mise à jour d'API parente, les constructeurs de ces classes attendaient un argument de type `Optional<Aggregator>` pour le sous-agrégateur par défaut (`defaultSub`). Or, les méthodes de copie `withSubs` et `withDefaultSub` transmettaient des objets `Aggregator` bruts, causant une erreur "incompatible types" à la compilation.

Solution : Envelopper explicitement le retour de la méthode `defaultSub()` dans un `Optional.of()`.

Avant (Parametric.java) :

```
@Override
public Parametric withSubs(Map<Criterion, Aggregator> newSubs) {
    return new Parametric(multiplied, weighting, newSubs, defaultSub());
}
```

Après (Parametric.java) :

```
@Override
public Parametric withSubs(Map<Criterion, Aggregator> newSubs) {
    return new Parametric(multiplied, weighting, newSubs, Optional.of(defaultSub()));
}
```

Note : La même modification exacte a été appliquée à la méthode `withDefaultSub` et dupliquée dans le fichier `Weighter.java`.

4 3. Erreur de méthode manquante (ifPresent)

Fichier concerné : `AggregatorJsonConverter.java`, lignes 105-108.

Problème (Pourquoi?) : Le code tentait d'appeler la méthode `.ifPresent()` sur le retour de `aggregator.defaultSub()`. Cependant, cette méthode renvoie désormais directement un objet `Aggregator` (et non plus un `Optional`), déclenchant l'erreur "cannot find symbol".

Solution : Remplacer la logique conditionnelle de l'`Optional` par un test d'égalité standard contre l'objet par défaut `Weighter.FULL_EQUAL_WEIGHTER`.

Avant :

```
aggregator.defaultSub().ifPresent(defaultSub -> node.set("defaultSub", toJsonNode(defaultSub)));
```

Après :

```
Aggregator def = aggregator.defaultSub();
if (!def.equals(Weighter.FULL_EQUAL_WEIGHTER)) {
    node.set("defaultSub", toJsonNode(def));
}
```

5 4. Prévention de la boucle infinie (StackOverflowError)

Fichiers concernés : `Weighter.java` (lignes 86-90), `Parametric.java`, `Owa.java`.

Problème (Pourquoi?) : Lors de l'exécution des tests, une erreur `StackOverflow` se produisait dans la méthode `equals()`. L'objet par défaut se comparait à lui-même de manière récursive à l'infini, saturant la pile d'exécution.

Solution : Ajout d'un court-circuit identitaire standard en début de méthode `equals` (`this == o2`) pour arrêter immédiatement la comparaison si les deux références pointent vers le même objet en mémoire.

Avant (`Weighter.java`) :

```
@Override
public boolean equals(Object o2) {
    if (!(o2 instanceof Weighter)) {
        return false;
    }
    // ... comparaison des attributs ...
}
```

Après (Weighter.java) :

```
@Override
public boolean equals(Object o2) {
    if (this == o2) return true; // Court-circuit salvateur
    if (!(o2 instanceof Weighter)) {
        return false;
    }
    // ... comparaison des attributs ...
}
```

Note : Appliqué également aux méthodes equals des classes Parametric et Owa.

6 5. Sensibilité linguistique des Tests

Fichier concerné : src/test/java/.../assessment/SchemaTests.java, lignes 36-39.

Problème (Pourquoi ?) : Le test unitaire validant le schéma JSON échouait sur un environnement d'exécution configuré en anglais (ex : WSL ou CI/CD Linux). Le test s'attendait strictement à un message d'erreur en français ("valeur dans l'énumération"), alors que la JVM renvoyait le message en anglais ("value in the enumeration").

Solution : Assouplissement des assertions pour tolérer les environnements francophones et anglophones en utilisant l'opérateur logique ||.

Avant :

```
assertTrue(e0.getMessage().contains("valeur_dans_l'énumération"), e0.getMessage());
Error e1 = errors.get(1);
assertTrue(e1.getMessage().contains("array_attendu"), e1.getMessage());
```

Après :

```
assertTrue(e0.getMessage().contains("enumeration") || e0.getMessage().contains("énumération"), e0.
    getMessage());
Error e1 = errors.get(1);
assertTrue(e1.getMessage().contains("array"), e1.getMessage());
```

7 6. Robustesse de la comparaison JSON

Fichier concerné : src/test/java/.../aggregator/JsonTests.java, lignes 14 et 65-68.

Problème (Pourquoi ?) : Le test testSerializeComplex validait la sérialisation JSON en comparant des chaînes de caractères brutes (String). Cette approche s'est révélée fragile à cause d'espaces invisibles ou de retours à la ligne discordants entre la sortie générée et le fichier attendu.

Solution : Utilisation de la bibliothèque Jackson (déjà présente dans le projet) pour parser les deux chaînes en JsonNode (arbres de données) et effectuer une comparaison structurelle insensible au formatage. Nous avons dû ajouter l'import de la classe ObjectMapper.

Avant :

```
assertEquals(expected.replaceAll("\\[\\n_+0.2,\\n_+0.8\\n_+\\]", "[_0.2,_0.8_]"),
    converter.toJson(tree).replaceAll("_:", ":").replaceAll("(?m)0\\.0$", "0").replace("{_}", "{}") + "
\\n");
```

Après (incluant le nouvel import en haut de fichier) :

```
import tools.jackson.databind.ObjectMapper; // Nouvel import ligne 14
// ...
// Remplacement à la ligne 65
String actual = converter.toJson(tree);
assertEquals(new ObjectMapper().readTree(expected), new ObjectMapper().readTree(actual));
```

8 7. Typage dans les instantiations de Test

Fichier concerné : `src/test/java/.../aggregator/OwaTests.java`, ligne 61.

Problème (Pourquoi ?) : Le test tentait d'instancier un `Owa` en passant un `Optional.empty()` alors que la signature statique `Owa.given()` exigeait des arguments classiques.

Solution : Simplification de l'appel pour utiliser la signature surchargée la plus simple de `Owa.given()` qui se charge déjà d'attribuer le bon comportement par défaut.

Avant :

```
private static Owa givenOwa(List<Double> weights) {
    return Owa.given(weights, ImmutableMap.of(), Optional.empty());
}
```

Après :

```
private static Owa givenOwa(List<Double> weights) {
    return Owa.given(weights);
}
```